

Date: 2018-06-04

Thomas Köppe <tkoeppe@google.com>

ISO/IEC JTC 1/SC 22/WG 21 P0806R2

To: CWG

Deprecate implicit capture of **this** via **[=]**

Contents

1. [Revision history](#)
2. [Summary](#)
3. [Discussion](#)
4. [Proposed wording](#)
5. [Future directions](#)
6. [Compatibility, upgrade path, warnings](#)

Revision history

- [P0806R0](#): Initial proposal.
- [P0806R1](#): Remove the **[&]** part; only deprecate **[=]** now, as requested by EWG. Rebased on current working paper wording.
- P0806R2: Minor word order changes in normative text; add example to Annex D entry.

Historical note: Revision 0 proposed to deprecate the implicit capture of **this** for both **[=]** and **[&]** *capture-defaults*. During discussion in Albuquerque, EWG wanted to retain **[&]** as an idiomatic way to capture **this** and only supported the deprecation the case of **[=]**.

Summary

We propose to deprecate the implicit capture of **this** in a lambda expression with a *capture-default* **[=]**. Users should explicitly use one of **[=, this]** or **[=, *this]**.

Before the proposal

```
struct Foo {
    int n = 0;
    void f(int a) {
        g([=](int k) { return
n + a * k; });
        g([=, *this](int k) {
return n + a * k; });
        g([&, a](int k) { n
+= a * k; });
    }
};
```

With the proposal

```
struct Foo {
    int n = 0;
    void f(int a) {
        g([=, this](int k) {
return n + a * k; });
        g([=, *this](int k) {
return n + a * k; });
        g([&, a](int k) { n
+= a * k; });
    }
};
```

Discussion

There are two kinds of default capture in a lambda expression:

- `[&]`: names in the lambda body refer to the actual entities that they name.
- `[=]`: names in the lambda body refer to local data members of the closure object that were copied from the entities in the surrounding scope.

When the lambda expression occurs inside a non-static class member function, any name `foo` of a non-static class member is interpreted as `this->foo`, where `this` is now bound to a local data member of the closure object. This copy is initialised from the value of `this` of the surrounding class member function *for both kinds of default capture*.

This situation is somewhat surprising and irregular. The semantics of capture-by-reference suggest that class members should refer directly to class members of the containing object. On the other hand, `this` is a pointer value, and it is clearly itself *copied* into the closure object. So whichever way one looks at the capture, one can rationalise for both `[&]` and `[=]` that they cause a copy of `this` to be stored in the closure object. By contrast, neither capture default effects the capture of `[*this]`, which would copy the entire containing object into the closure object and rebind `this` to point to that copy.

At this point, some historic background is helpful. The `this` keyword was introduced to C++ before references. At the time, C++ was translated into C,

and the simple pointer semantics of `this` were convenient. If the entire feature set of references, value categories and classes had been designed together, `this` would have been an lvalue designating the object, and not a prvalue designating the address of the object. We can adopt this historic perspective by thinking of the fundamental object as “`*this`” rather than `this`. Let us consider how we might capture this object:

- Capture `*this` by value: `[*this]`
- Capture `*this` by reference: hypothetically we would say “`[&*this]`”; in C++ we say `[this]`
- If the default capture is reference capture, the hypothetical “`[&, &*this]`” (in real C++: `[&, this]`) is a redundant reference capture; `[&, *this]` is a non-redundant value capture.
- If the default capture is value capture, the hypothetical “`[=, &*this]`” (in real C++: `[=, this]`) is a non-redundant reference capture; `[=, *this]` is a (redundant-looking) value capture.

In C++14, the fact that both default captures captured the `this` pointer was perhaps peculiar, but unambiguous. But C++17 added genuine value-capture of the containing object via `[*this]`, so that it is now more surprising that `[=]` means `[=, this]` and not `[=, *this]`. In other words, one default capture (`[&]`) captures `*this` in the way that would be redundant when spelled out, but the other capture (`[=]`) captures it in the non-redundant way.

This inconsistency in defaults is confusing. Users may well know that there *exists* an inconsistency, but it is much harder to know which way round the inconsistency goes. For this reason, we propose that users should never rely on implicit capture of `*this` via a `[=]`-capture-default. (The implicit capture of `*this` via `[&]` continues to be idiomatic. An earlier revision of this paper also proposed its deprecation.)

- `[=]` → `[=, this]`: local variables by value, class members by reference
- `[=]` → `[=, *this]`: everything by value
- `[&]` → `[&, this]`: everything by reference
- `[&]` → `[&, *this]`: (this would be unusual)

Beyond this proposal, we recommend that as a matter of style, code should only ever capture `*this` explicitly, even in the presence of *capture-defaults*.

Proposed wording

Append a sentence to [8.4.5.2 `expr.prim.lambda.capture`]p7:

[...] If an expression potentially references a local entity within a declarative region in which it is odr-usable, and the expression would be potentially evaluated if the effect of any enclosing `typeid` expressions ([`expr.typeid`, 8.5.1.8]) were ignored, the entity is said to be *implicitly captured* by each intervening *lambda-expression* with an associated *capture-default* that does not explicitly capture it. The implicit capture of `*this` is deprecated when the *capture-default* is `=`; see [`depr.capture.this`, D.?]. [Example: [...]]

Insert a new section [D, depr], between the current [D.3, depr.except.spec] and [D.4, depr.cpp.headers].

D.? Implicit capture of `*this` by reference **[`depr.capture.this`]**

For compatibility with prior C++ International Standards, a lambda expression with *capture-default* = ([8.4.5.2, `expr.prim.lambda.capture`]) can implicitly capture `*this` by reference. [Example:

```
struct X {  
    int x;  
    void foo(int n) {  
        auto f = [=]() { x = n; }; // deprecated: x  
means this->x, not a copy thereof  
        auto g = [=, this]() { x = n; }; // recommended  
replacement  
    }  
};
```

— end example]

Future directions

The obvious next step would be to make the implicit capture of `*this` ill-formed in a `[=]`-capture. This could only happen post-C++20, since valid C++17 code must first upgrade to C++20 by changing `[=]` to `[=, this]`, which is ill-formed in C++17 (added by [P0409R2](#)). The meaning of `[=]` could then be changed again in the farther future. Such a change would presumably affect a large amount of use cases, but the upgrade path is straight-forward. Alternatively, the feature

could simply remain deprecated to guide users and authors of style guides to avoid implicit capture of `*this` in new code.

Compatibility, upgrade path, warnings

Let us briefly examine the upgrade path and compiler warnings that would likely result from this proposed change, in the spirit of [P0684R2](#). The change does not break otherwise valid C++20 code, and the earliest revision in which a breakage from C++17 could appear is C++23. Therefore compilers for the current standard (C++17) need not emit *any* warning, but at best it would be a “future deprecation” warning rather than a “future breakage” warning. In C++20 conformance mode, compilers may reasonably emit a deprecation warning by default, or they could defer any warnings to the “future breakage” kind if and when a proposal for a breaking change is accepted. The replacement `[=, this]` for the deprecated `[=]` is ill-formed in C++17 and only available in C++20 as of [P0409R2](#).